

[← Go Back](#)

Hardware

GIGA Display Shield

Tutorials

[Getting Started with the GIGA Display Shield](#)[GIGA Display Shield GFX Guide](#)[GIGA Display Shield LVGL Guide](#)[GIGA Display Shield Draw Images Guide](#)[GIGA Display Shield Touch Interface](#)[GIGA Display Shield Automatic Screen Orientation with IMU](#)[GIGA Display Shield Microphone Guide](#)[GIGA Display Shield Camera Connector Guide](#)[GIGA Display Shield 3D Engine Tutorial](#)[SquareLine Studio With the GIGA Display Shield](#)[Display Camera Feed on GIGA Display Shield with OpenMV](#)

GIGA Display Shield emWin Guide

[GIGA Display Shield AppWizard Guide](#)[GIGA Display Shield Display Image From USB](#)[Home](#) / [Hardware](#) / [GIGA Display Shield](#) / [GIGA Display Shield emWin Guide](#)

GIGA Display Shield emWin Guide

Learn how to use the emWin library with the GIGA Display Shield.

Author · Benjamin Dannegård · Last revision · 12/05/2024

Introduction

Segger's emWin is a graphical framework for building powerful UIs, and is fully compatible with the GIGA Display Shield. It allows you to build UIs, using pre-made widgets like buttons, images, loading bars, sliders, checkboxes, etc. It also allows you to fully customize the screenspace on the display. In this guide, we will go through some of the different components, so you can learn how to best implement it in your projects.



emWin full demo running on the GIGA Display Shield

ON THIS PAGE

Introduction

[Hardware & Software Needed](#)[Downloading the Library and Core](#)[emWin Library Setup](#) +[Visual Elements](#) +[Functional Elements](#) +[Conclusion](#)[Next Step](#)

Hardware & Software Needed

[Arduino GIGA R1 WiFi](#)[Help](#)

Downloading the Library and Core

The GIGA R1 core includes the [Arduino_H7_Video](#) library that handles the display.

In this guide, we will be using three different libraries:

[Arduino_H7_Video](#), this one is bundled with the core, so make sure you have the latest version of the [Mbed core](#) installed.

[Arduino_GigaDisplayTouch](#), handles touch on the GIGA Display Shield

[emWin-Arduino-Library](#)

To install them, open the library manager and install the latest version by searching for "**emWin**" and "**Arduino_GigaDisplayTouch**".

In the sketch include the emWin library like this:

```
1 #include "DIALOG.h"
```

There is no need to include the "**Arduino_GigaDisplayTouch**" library separately, as it is already incorporated within the emWin library.

emWin Library Setup

In this section, we will go through

How to define & configure the display,
how to create a grid layout,
how to add an object to the grid,
how to update the display.

emWin Screen Configuration

When creating elements, information about the screen and placement needs to be provided. The provided code initializes the emWin library, configuring display and touch controller settings. It sets the display to landscape mode, enables multi-buffering for the Windows manager, and creates a main window for content rendering.

Full Example:

```
1 void setup() {  
2   delay(3000);  
3   /* Init SEGGER emWin l  
4   GUI_Init();  
5  
6   LCD_ROTATE_SetSel(1);  
7   WM_MULTIBUF_Enable(1);  
8  
9   /* Create the main win  
10  WM_CreateWindowAsChild  
11 }
```

Window Manager

The examples in this tutorial will be using the window manager method. This makes it easier to manage the screen elements and widgets. However, it is also possible to create emWin sketches without the window

Here is an example that sets the screen background color to green and prints the text "Hello world" in red on the screen. Lets first take a look at how this would look without the window manager:

```
1 #include <DIALOG.h>
2
3 void setup() {
4     delay(3000);
5     GUI_Init();
6     GUI_MULTIBUF_Begin();
7     GUI_SetBkColor(GUI_GRE
8     GUI_Clear();
9     GUI_SetColor(GUI_RED);
10    GUI_DispString("Hello
11    GUI_MULTIBUF_End();
12 }
13
14 void loop() {
15     GUI_Exec();
16 }
```

And to get the same result with a window manager, it would look like this:

```
1 #include "DIALOG.h"
2
3 static void _cbWin(WM_ME
4     switch (pMsg->MsgId) {
5     case WM_PAINT:
6         GUI_SetBkColor(GUI_G
7         GUI_Clear();
8         GUI_SetColor(GUI_RED
9         GUI_DispString("Hell
10        break;
11    default:
12        WM_DefaultProc(pMsg)
13        break;
14    }
15 }
16
17 void setup() {
18     GUI_Init();
19     WM_MULTIBUF_Enable(1);
20     WM_CreateWindowAsChild
21 }
22
23 void loop() {
24     GUI_Exec();
25 }
```

Using the window manager method is more complex and require more lines of code, but this method helps with keeping your GUI structured as it gets more elements and widgets to display. For example with the window manager it is impossible to overwrite areas outside a windows designated area. For simpler applications, the window manager may not be needed. Keep this in mind when developing your own emWin solutions! For more information, navigate over to the [emWin on Arduino documentation](#).

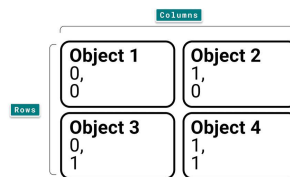
Creating a Grid Layout

To create different windows inside our display, use the

```
WM_CreateWindowAsChild()
```

```
1 WM_CreateWindowAsChild(X-
```

Grid Layout (emWin)



Grid layout infographic

This is how it would look to use emWin to create a 2x2 grid layout inside the main window:

Full Example:

```
1 static void _cbWin(WM_ME
2     switch (pMsg->MsgId) {
3         case WM_CREATE:
4             /* [0, 0] - Image
5             WM_CreateWindowAsC
6
7             /* [1, 0] - Slider
8             WM_CreateWindowAsC
9
10            /* [0, 1] - Checkb
11            WM_CreateWindowAsC
12
13            /* [1, 1] - Progre
14            WM_CreateWindowAsC
15            break;
16        case WM_PAINT:
17            GUI_SetBkColor(0x0
18            GUI_Clear();
19            break;
20        default:
21            WM_DefaultProc(pMs
22            break;
23    }
24 }
```

In the loop of any emWin sketch, the bare minimum that needs to be called is `GUI_Exec()`. This will keep the engine alive, handle touch events for the rest of the sketch among other things.

```
1 void loop(){  
2   GUI_Exec();  
3 }
```

Visual Elements

Image

To be able to display an image it first needs to be defined.

bmarduinologo is the filename of the image. The image file needs to be in the same folder as the sketch. The image needs to be referenced like this:

```
1 extern GUI_CONST_STORAGE
```

To convert images into the emWin format, you can use the [emWin Bitmap Converter](#) tool.

Now the image can be drawn by specifying the image file along with the x and y position. Like this:

```
1 GUI_DrawBitmap(&bmarduino
```



Displaying an image with
emWin

Full Example:

Remember that the image file needs to be in the same folder as the sketch, use the image that comes with the full demo (File > Examples > Arduino_H7_Video > emWinDemo).

```

1  #include "DIALOG.h"
2
3  extern GUI_CONST_STORAGE
4
5  static void _cbWin(WM_M
6      switch (pMsg->MsgId)
7      case WM_CREATE:
8          break;
9      case WM_PAINT:
10         GUI_SetBkColor(GU
11         GUI_Clear();
12         /* Draw image */
13         GUI_DrawBitmap(&b
14         break;
15     default:
16         WM_DefaultProc(pM
17         break;
18 }
19 }
20
21 void setup() {
22     /* Init SEGGER emWin
23     GUI_Init();
24
25     LCD_ROTATE_SetSel(1);
26     WM_MULTIBUF_Enable(1)
27
28     /* Create the main wi

```


Most widgets have label functions attached to them. However if you want to print something independent of a widget you can use the `GUI_DispString("")` function. This will display any text put in as a parameter.

Calling `GUI_SetColor()` before the text print will allow you to change the color of the text that is then printed. [Here](#) you can find the colors available.

For changing the font size, use `GUI_SetFont(&GUI_Font16_1);`, this call will set the font size to 16.

Full Example:

```
1  #include "DIALOG.h"
2
3  static void _cbWin(WM_MESSAGE) {
4      switch (pMsg->MsgId) {
5          case WM_PAINT:
6              GUI_SetBkColor(GUI_WHITE);
7              GUI_Clear();
8              GUI_SetColor(GUI_RED);
9              GUI_SetFont(&GUI_Font16_1);
10             GUI_DispString("Hello World!");
11             break;
12         default:
13             WM_DefaultProc(pMsg);
14             break;
15     }
16 }
17
18 void setup() {
19     delay(3000);
20     GUI_Init();
21     WM_MULTIBUF_Enable(1);
22     WM_CreateWindowAsChild(WM_DIALOG, 0, 0, 100, 100, WM_WHITE, WM_BLACK);
23 }
24
25 void loop() {
26     GUI_Exec();
27 }
```

Functional Elements

Checkbox

Like with other widgets this line creates the checkbox widget:

```
1 hBox = CHECKBOX_CreateEx(
```

If we want a label next to the checkbox, use

```
CHECKBOX_SetText(hBox,  
"Check");
```

. Where you specify the checkbox the text belongs to and what the text should say. The font and color can then be set for the text with the following lines of code:

```
1 CHECKBOX_SetText(hBox, "C  
2 CHECKBOX_SetTextColor(h  
3 CHECKBOX_SetFont(hBox,
```

Then for the state of the checkbox the number of states and where it should start will be determined by these lines:

```
1 CHECKBOX_SetNumStates(hBo  
2 CHECKBOX_SetState(hBox,
```





emWin checkbox
unchecked

Full Example:

```

1  #include "DIALOG.h"
2
3  static void _cbChildWin
4      static WM_HWIN  hBox;
5      BUTTON_Handle  hButt
6      int            NCode
7      char           acBuf
8      int            State
9      static int      Click
10
11  switch(pMsg->MsgId) {
12      case WM_CREATE:
13          /* Create CHECKBO
14          hBox = CHECKBOX_C
15          /* Edit widget pr
16          CHECKBOX_SetText(
17          CHECKBOX_SetTextC
18          CHECKBOX_SetFont(
19          /* Set number of
20          CHECKBOX_SetNumSt
21          /* Manually set t
22          CHECKBOX_SetState
23          break;
24      case WM_PAINT:
25          GUI_SetBkColor(GU
26          GUI_Clear();
27
28          /* Display curren

```

Slider

Like the other widgets this line of code creates it:

```
1 hSlider = SLIDER_CreateEx
```

The specifications of the slider, such as range, number of ticks marks, value and width of the marker are set by the following lines:

```
1 SLIDER_SetRange(hSlider,  
2 SLIDER_SetNumTicks(hSl  
3 SLIDER_SetValue(hSlider  
4 SLIDER_SetWidth(hSlider
```

If you want to display the sliders value adjacent to the slider, use the following lines:

```
1 Value = SLIDER_GetValue(h  
2 sprintf(acBuffer, "Valu  
3 GUI_DisStringAt(acBuff
```

`sprintf()` will print the value as text and `GUI_DisStringAt()` will decide where the text is printed.

The GUI will also have to be re-drawn when the value changes so the display stays accurate. This can be achieved with:

```
1 case WM_NOTIFICATION_VALU  
2 WM_InvalidateWindow(p  
3 break;  
4 }  
5 break;
```



emWin slider

Full Example:

```

1  #include "DIALOG.h"
2
3  static void _cbChildWin
4      static WM_HWIN hSlide
5      int          NCode,
6      int          Value;
7      char         acBuff
8
9      switch(pMsg->MsgId) {
10         case WM_CREATE:
11             /* Create horizon
12             hSlider = SLIDER_
13             /* Set range of s
14             SLIDER_SetRange(h
15             /* Set number of
16             SLIDER_SetNumTick
17             /* Set value of s
18             SLIDER_SetValue(h
19             /* Set width of t
20             SLIDER_SetWidth(h
21             break;
22         case WM_PAINT:
23             GUI_SetBkColor(GU
24             GUI_Clear();
25             GUI_SetFont(&GUI_
26             GUI_SetColor(GUI_
27
28             /* Display slider
29             GUI_SetValue(hSlider,

```

Progress Bar

Creating the progress bar is similar to the other widgets:

```
1 PROGBAR_Handle hProg;  
2 hProg = PROGBAR_CreateEx(
```

The progress bar needs a callback function that will handle the calculation and animation of the progress that is being displayed. The progress bar and callback function that should be linked are defined in the `WM_SetCallback()` function, like so:

```
1 WM_SetCallback(hProg, _cb
```

To display the progress on the progress bar, a rectangle will have to be drawn inside of the existing progress bar, like this:

```
1 WM_GetClientRect(&Rect);  
2 GUI_SetColor(GUI_BLACK)  
3 GUI_AA_DrawRoundedRectE
```

The progress bars value, or the progress of it can be calculated and displayed with these lines:

```
1 Value = PROGBAR_GetValue(  
2 ValueF = Value / 100.0F  
3 sprintf(acBuffer, "Prog
```

In the `void loop()` of the sketch the calculation of time for the

also be set. Using

```
PROGBAR_SetValue(progressbar,  
progressbarcount)
```

the value of the progress bar can be set from the `loop()`. Then it can be redrawn to show how it updates.

```
1  if (millis() - previousMi  
2      previousMillis = mill  
3      progbarCnt++;  
4      if (progbarCnt > 100)  
5          progbarCnt = 0;  
6      }  
7      PROGBAR_SetValue(hPro  
8      WM_InvalidateWindow(h  
9  }
```



Full Example:



```

1  #include "DIALOG.h"
2
3  PROGBAR_Handle hProg;
4
5  static void _cbChildWin
6      GUI_RECT Rect;
7      float    ValueF;
8      int      Value;
9      char     acBuffer[16]
10
11  switch (pMsg->MsgId)
12      case WM_CREATE:
13          hProg = PROGBAR_C
14          WM_SetCallback(hP
15          break;
16      case WM_PAINT:
17          GUI_SetBkColor(GU
18          GUI_Clear();
19          break;
20      default:
21          WM_DefaultProc(pM
22          break;
23  }
24 }
25
26 static void _cbProgbar(
27     GUI_RECT Rect;
28     float    ValueF;

```

Button

Like the other widgets, this one is created with:

```
1 hButton = BUTTON_CreateEx
```

And to set the text on the button, use:

```
1 BUTTON_SetText(hButton, "
```

To check for the button being

`Clicked` and `Released`. Then simply put these inside the `if()` statements to make them trigger whenever the button is pressed or released.

```
1 static int      Clicked,  
2  
3 if(Clicked) {  
4     sprintf(acBuffer, "But  
5     GUI_DispStringAt(acBuf  
6 }  
7 if(Released) {  
8     sprintf(acBuffer, "But  
9     GUI_DispStringAt(acBuf  
10 }
```



emWin button



emWin button clicked

Full Example:



```

1  #include "DIALOG.h"
2
3  #define ID_BUTTON 1
4
5  static void _cbChildWin
6      static WM_HWIN  hBox;
7      BUTTON_Handle  hButt
8      int             NCode
9      char            acBuf
10     int             State
11     static int      Click
12
13     switch(pMsg->MsgId) {
14         case WM_CREATE:
15             /* Create a butto
16             hButton = BUTTON_
17             BUTTON_SetText(hB
18             break;
19         case WM_PAINT:
20             GUI_SetBkColor(GU
21             GUI_Clear();
22             GUI_SetFont(&GUI_
23             GUI_SetColor(GUI_
24
25             /* Check button s
26             if(Clicked) {
27                 sprintf(acBuffe
28                 GUI_DisStringA

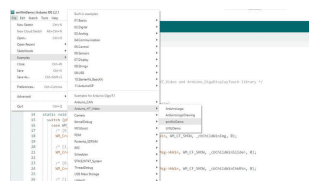
```

Conclusion

This guide went through the building blocks of the different components that can be implemented with emWin. To see these examples in a full running example sketch go to

File > Examples >

Arduino_H7_Video > emWinDemo.



Example in the IDE

This demo sketch will show the

Next Step

emWin is a comprehensive library and GUI framework that has a lot of customizability, if you are interested in playing around more with it, you can find many different examples and widgets on the official website for [Segger® emWin](#). The code on the website can easily be adapted into a sketch for the GIGA Display Shield.

Suggest changes	Need support?	License
The content on docs.arduino.cc is facilitated through a public GitHub repository . If you see anything wrong, you can edit this page here .	Help Center Ask the Arduino Forum Discover Arduino Discord	The Arduino documentation is licensed under the Creative Commons Attribution-Share Alike 4.0 license.

Was this article helpful?

